AuraTrack 2.0

By: Cesar Picazo

Original Project Summary

- Built a Music Recommender Simulation modeled after real-world systems like Spotify
- Used numerical methods to compare songs against a user profile and return the top-k most similar tracks
- Goal was to understand the structure of AI-assisted recommendation through a simulated pipeline

What I have Built

- Expanded the dataset to 550,000+ songs for real-world scale
- Added Gaussian Mixture Model (GMM) for candidate generation
- Integrated Claude API (LLM) for self-critique — evaluates whether recommendations logically match the user profile
- Implemented adaptive weights — if reliability score drops below 0.70, a second LLM call adjusts the scoring formula per user
- Wrapped everything in an agentic loop that ties profile setup, retrieval, ranking, and critique into one end-to-end pipeline

Demo #1: User Profile Init

Input:

```
favorite_artists = ["Taylor Swift", "The Weeknd", "Dua Lipa"]
favorite_genres = ["Pop", "Rock", "Jazz"]
```

Output:

```
Generated User Profile:
 {
  "preferred_energy": 0.72,
  "preferred_acousticness": 0.25,
  "preferred_valence": 0.65,
  "preferred_tempo": 118.0,
  "preferred_danceability": 0.7,
  "preferred_speechiness": 0.08,
  "preferred_loudness": -5.5,
  "preferred_liveness": 0.18
 }
```

Demo #2: Explanation Generation

Prompt:

```
prompt = f"""Given the following ranked recommendations with their feature contributions, generate a plain English expl
Focus on the most influential features and how they align or misalign with the user's preferences. Make the explanation
Here are the recommendations with scores and contributions: {recommendations_structured}.
Here is the user profile: {user_prefs.structure_profile()}.
Respond with a JSON with no preamble, one per recommended song, in the
same order as the input recommendations:
{{"song_id": "explanation"}} with all recommended song IDs as keys and the generated explanations as values.
"""
```

Output:

```
Song ID: 123
Explanation: Song A landed in the top spot because it perfectly matches your love for high-energy Pop music. With its upbeat tempo around 130
BPM and great danceability, it aligns well with your preference for energetic, dance-friendly tracks. The loud, vibrant production style also
matches what you typically enjoy.

Song ID: 456
Explanation: Song B came in second as a solid Rock track with strong energy that matches your taste. While it has the energetic vibe you like,
 it's slightly quieter and less danceable than your usual preferences, which is why it ranked just below Song A. Still, it fits well with your
 appreciation for Rock music.

Song ID: 789
Explanation: Song C ranked third because while you do enjoy Jazz, this track is more mellow and acoustic than your typical preferences. It has
 lower energy and danceability compared to what you usually gravitate toward, and its quieter, more laid-back feel differs from your preferred
 upbeat style. However, it still made the list due to your interest in the Jazz genre.
```

Demo #3: Weight Change

Prompt:

```
prompt1 = f"""
Given top 5 recommendations with the scores, and user profile, check if the rankings make
logical sense given the user's preferences. Flag any contradictions and produce a reliability
score (0-1) for the overall recommendation set such as a high energy song ranking highly for a
low energy user because tempo dominated.
Here are the recommendations with scores: {recommendations_structured}.
Here is the user profile: {user_prefs_structured}.
Respond in JSON format with no preamble with keys "reliability_score" (float between 0 and 1) and
"contradictions" (list of strings describing any contradictions found, empty if none found).
"""
```

Output:

```
Old Weights:
 {
  "energy": 0.22,
  "acousticness": 0.18,
  "valence": 0.13,
  "tempo": 0.1,
  "danceability": 0.08,
  "loudness": 0.08,
  "liveness": 0.06,
  "speechiness": 0.05,
  "genre": 0.06,
  "artist": 0.02,
  "title": 0.02
 }
New Weights:
 {
  "energy": 0.28,
  "acousticness": 0.24,
  "valence": 0.13,
  "tempo": 0.1,
  "danceability": 0.08,
  "loudness": 0.08,
  "liveness": 0.06,
  "speechiness": 0.05,
  "genre": 0.06,
  "artist": 0.02,
  "title": 0.0
 }
Warning: adjusted weights sum to 1.100, not 1.0
```

Reflection

- **Limits:** Data doesn’t have full coverage.
- **Learned:** AI’s unreliability
- **Future Updates:** Create full-working platform of multiple users with better generation and data.

Thank you!

Code link: https://github.com/cpicazo8304/applied-ai-system-project

What this project says about me as an AI engineer.

This project reflects my ability to utilize machine learning algorithms and AI techniques like connecting clustering, scoring, and language models into a cohesive, production-inspired pipeline. Not only that, it shows that I don't just implement techniques, but ask why they belong together and how they make the output more trustworthy. Building the self-critique loop in particular demonstrates that I think about reliability and failure modes, not just accuracy. I'm drawn to the intersection of classical ML and modern LLMs, and this project shows I can bridge both in a meaningful way. Most importantly, it reflects an engineering mindset, which is when given a real problem and a large messy dataset, I can design and build toward something that actually works.